

第15章 继承和派生

15.1 黑盒复用和白盒复用

复用既有代码最常见的方式是通过拷贝、粘贴并修改代码,但这种方式会破坏原有类的完整性,容易引入新的缺陷,且从严格意义上讲,属于重新编写代码。因此,这种“复制-修改”的做法并不属于通常意义上的软件复用。真正意义上的软件复用,是指在不改动现有代码的基础上,通过扩展机制来复用已有代码的功能与实现。这样的软件复用一般又分为两类:黑盒复用和白盒复用。

15.1.1 黑盒复用

黑盒复用是指通过既有代码的外部可见接口访问既有代码的方式,扩展并复用既有代码。在此过程中,无需知道既有代码的内部数据和具体实现,可见黑盒复用是对既有代码的功能复用。在面向对象程序设计中,可以通过依赖、普通关联、组合、聚合等逻辑关系,扩展并复用既有的类,此时的复用只用到类的公有接口,因此这种复用属于黑盒复用。例如,下面代码以黑盒复用的方式复用既有的 Book 类:

```
1  class Book {
2  public:
3      int countPages( ) const;
4      //其它略
5  };
6  //使用依赖复用 Book 类
7  class Person {
8  public:
9      void readBook(Book & aBook) { cout << aBook.countPages( ); }
10 };
11 //使用普通关联复用 Book 类
12 class Person {
13 public:
14     Person( Book & book ): mpBook( &book ) { }
15     void readBook( ) { cout << mpBook->countPages( ); }
16 private:
17     Book * mpBook;
18 };
19 //使用组合复用 Book 类
20 class BookShelf {
21 public:
```

```

22     void addBook( ) { mBooks.push_back(new Book); }
23     void removeBook( Book * pBook) { mBooks.remove(pBook); delete pBook; }
24     void read( ) { for( auto book: mBooks)    book->countPages( );    }
25     ~BookShelf( ) { for( auto book: mBooks)    delete book; mBooks.clear( ); }
26 public:
27     std::list<Book *> mBooks;
28 };
29 //使用聚合复用 Book 类
30 class BookShelf {
31 public:
32     void addBook(Book & book) { mBooks.push_back( &Book); }
33     void removeBook( Book * pBook) { mBooks.remove(pBook); }
34     void read( ) {
35         for( auto book: mBooks)    book->countPages( );
36     }
37 public:
38     std::list<Book *> mBooks;
39 };

```

黑盒复用是经常使用的复用技术，它有几个特点：

1. 黑盒复用是一种功能复用。
2. 实现黑盒复用时，可通过类(对象)间的水平关系-普通关联、聚合、组合和依赖等扩展被复用类。
3. 改变被复用类的具体实现，不影响客户端的既有代码。如修改了 `Book::countPages( )` 的具体实现，或者增加了 `Book` 的私有成员，都无需修改 `Person`、`BookShelf` 的类定义和实现，就能使用 `Book` 类中新的功能实现。
4. 由于是功能复用，复用时不需要被复用类的完整源码，只要求被复用类能够实例化，同时知道类的外部访问接口，无须知道类中各成员的具体实现。这点对复用二进制代码非常有用，例如，对于很多闭源的软件，虽然没有完整源码，但客户仍能使用其软件功能。
5. 黑盒复用要求被复用类具有良好的设计，这里的良好设计是指类的访问接口具有一定的完备性和通用性，也就是要求访问接口是稳定的、充分的。但给出完美设计是困难的，甚至理论上就不可行，因为这涉及到被复用类的适用范围、行为的合理分组、各行为的独立性、各行为的合理性等多方面。因此，良好的设计应保证接口在一定的适用范围内具有稳定性，选择和确定这个适用范围是设计中的重要内容。

15.1.2 白盒复用

白盒复用是指通过复用完整的既有代码进行扩展。在此过程中，被复用的既有代码是透明的，其具体内容和实现细节对复用者来说均是可见的，也因此称为白盒复用。不同于黑盒复用的功能复用，白盒复用是一种代码复用或实现复用。在面向对象程序设计中，白盒复用是使用继承的方式实现的。例如，下面代码以继承的方式，白盒复用既有的 `Book` 类：

```
1  class Book {
2  public:
3      int countPages( ) const;
4      //其它略
5  };
6  class Cartoon:public Book {
7  public:
8      float price( ) { return countPage( ) * 0.1; }
9  };
```

15.2 继承

在 C++ 语言中，定义继承关系的语法格式为：

```
class/struct 派生类类名 : [继承方式] 基类名称 [, [继承方式] 基类名称 ] {
    类定义内容
};
```

其中冒号后的部分称为继承列表，指明派生类的多个基类名称及各自的继承方式；继承方式可以是 `public`、`protected` 或者 `private`；如果省略继承方式，那么对于 `class`，缺省的继承方式为 `private`，对于 `struct`，缺省的继承方式为 `public`。例如：

```
1  class Child : public Parent1, Parent2 {
2  public:
3      Child( int n);
4      void childFunc( );
5  private:
6      int          mData;
7      OtherClass * mpOther;
8  };
```

这里派生类 `Child` 继承自 `Parent1` 和 `Parent2` 两个基类，也可以说从 `Parent1` 和 `Parent2` 两个基类派生出 `Child` 类，并且 `Child` 以公有继承方式继承 `Parent1`，以私有继承的方式继承 `Parent2`。

在继承关系中，基类的数量可以是一个或多个。如果派生类只有一个基类，称为单继承；若有

多个基类，称为多(重)继承。

基类派生出派生类后，派生类可继续派生，这样可以定义出具有多个层次的继承树，也称多层继承。在实践中，多层继承的继承层次应控制在 3-5 层以内，至多 7-9 层；过多的继承层次会大大增加维护、理解继承树的困难程度，一旦层次过多，更应怀疑是设计出现了问题。

从基类派生时，可以选择不同的继承方式，特别地，在 `public` 继承方式下，基类也可称为父类，派生类可称为子类。继承关系的 UML 图例如图 15-1。

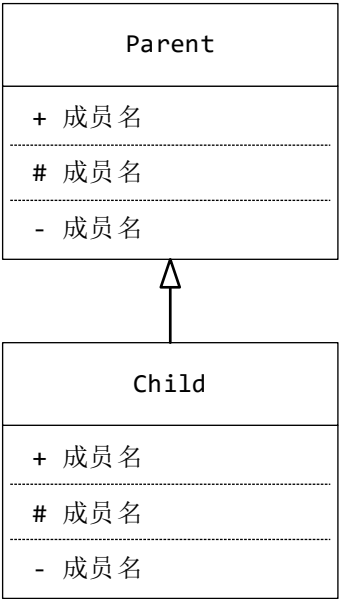


图 15-1 继承关系的 UML 图例

15.2.1 派生类中的成员

派生类是通过继承关系复用基类的代码和实现来定义的，是新定义的一个类型，本质上派生类和基类是两个类型，也就是说使用继承关系，本质上只是简化定义新类型的过程和代码量，因此，普通类具有的特点和语法要求，派生类也同样具有，例如派生类也有构造、拷贝、赋值、析构等函数，派生类中成员也有访问控制等。派生类中的成员包括：

- (1) 派生类的构造、拷贝、赋值、析构函数。
- (2) 派生类中新增的成员函数和数据成员。
- (3) 继承自基类的，即基类中除构造、拷贝、赋值、析构、自动转换函数之外的其它所有成员。

例如：

```
1 class A {  
2 public:
```

```

3     void aFunc1( );
4     void aFunc2( );
5 private:
6     void aFunc3( );
7     static void fA( );
8 private:
9     int mData;
10    static int sDataA;
11 };
12 //以继承方式复用类 A 的实现
13 class B : public A {
14 public:
15     void bFunc1( ) {
16         bFunc3( );
17         aFunc1( );
18     }
19     void bFunc2( );
20     static void fB( );
21 private:
22     void bFunc3( );
23 private:
24     int mNum;
25     static int sDataB;
26 };

```

那么，派生类 B 中的成员有：

类 B 中缺省的：

构造、析构、拷贝、赋值函数

类 B 中定义的：

B::bFunc1()

B::bFunc2()

B::fB() //类方法

B::bFunc3()

B::mNum

B::sDataB //类变量

从类 A 继承的：

A::aFunc1()

A::aFunc2()

A::aFunc3()

A::fA() //类方法

A::mData

A::sDataA //类变量

实例化的派生类对象与普通对象一样，对象中只存放实例变量，不存放成员函数和类变量。例

如，实例化上边的类 B，那么类 B 的对象中只有数据 A::mData 和 B::mNum。由于派生类对象中含有完整的基类的数据，所以也可以说派生类对象中包含基类对象，或者说包含基类子对象。

对于派生类对象的内存布局，应满足基类子对象位于低地址，派生类数据位于高地址，即存放顺序是基类数据在前，派生类数据在后。单继承时，派生类的地址就是基类子对象的地址；在多重继承时，由于派生类对象中有多个基类子对象的存在，派生类的地址只是第一个基类子对象的地址，不是其它基类子对象的地址。对比这两种内存布局，容易看出，若希望从派生类对象的地址得到基类子对象的地址，单继承下简单高效，多重继承下则需要一些额外的内部操作。

15.2.2 继承下的访问控制

派生类中的成员有来自本类的，还有来自基类的，而且每个成员都有自己的访问控制。对于来自本类的成员，其访问控制就是在派生类中指定的访问控制；对于来自基类的成员，其访问控制遵循表 15-1 中的规则：

表 15-1 不同继承方式下基类成员在派生类中的访问控制

在派生类中的访问控制 继承方式	基类成员在基类中的访问控制		
	public	protected	private
public	public	protected	不可访问
protected	protected	protected	不可访问
private	private	private	不可访问

从表中可看出，基类中的私有成员在派生类中都是不可访问的；使用私有继承方式，基类中的公有和保护成员在派生类中具有私有的访问控制，若继续派生，都将成为不可访问的。因此，在多层继承时，使用私有继承，会导致顶层类中成员的可访问性至多传递一层，而若使用公有继承或保护继承，顶层类中公有和保护成员的可访问性可以沿着继承树向下传递。

其中，protected 访问控制就是为了适应继承而引入的。对于单个类来说，protected 访问控制等同于 private 访问控制，也是类外不可访问；在多层继承中，若采用 public 继承或 protected 继承方式，那么顶层类中具有 protected 访问控制的成员，在所有的后裔类中仍具有 protected 访问控制。也就是说，具有 protected 访问控制的成员，在 public 或 protected 继承方式下，对派生类可见，但对其它类不可见。例如：

```
1 class Base {
2     public:
3         Base( ) = default;
4         void funcPub ( ) { }
5     protected:
```

```
6     void funcProt( ) { }
7     int mNum2;
8     private:
9         void funcPriv( ) { }
10        int mNum1;
11    };
12    class Derived : public Base {
13    public:
14        Derived( );
15        void g1( ) { }
16    protected:
17        void g2( ) { }
18        int mVal2;
19    private:
20        void g3( ) { }
21        int mVal1;
22    };
23    //外联实现
24    Derived::Derived( ) {
25        funcPub( ); //合法
26        funcProt( ); //合法
27        funcPriv( ); //非法
28        mNum2 = 5; //合法
29        mNum1 = 6; //非法
30        g1( ); //合法
31        g2( ); //合法
32        g3( ); //合法
33        mVal2 = 10; //合法
34        mVal1 = 20; //合法
35    }
36    int main( ) {
37        Derived d;
38        d.funcPub( ); //合法
39        d.funcProt( ); //非法
40        d.funcPriv( ); //非法
41        d.mNum2 = 10; //非法
42        d.mNum1 = 20; //非法
43        d.g1( ); //合法
44        d.g2( ); //非法
45        d.g3( ); //非法
46        d.mVal2 = 88; //非法
47        d.mVal1 = 99; //非法
48    }
```

15.3 派生类的构造、析构、拷贝、赋值

在派生类中，可以自定义构造函数、析构函数、拷贝构造函数和赋值函数。如果未自定义，编译器将提供缺省的相应函数。需要注意的是，基类的构造函数、析构函数、拷贝构造函数、赋值函数以及类型转换函数等不会自动继承到派生类中。例如，即使基类中定义了自定义构造函数，若派生类没有显式定义构造函数，则会使用编译器提供的缺省构造函数，而不是基类中的自定义构造函数。若希望派生类就使用基类的构造函数，可以通过 `using Base::Base;` 这样的语句来实现继承，但这里不展开详细讨论。

15.3.1 构造和析构

在构造派生类时，必须先构造基类，然后再构造派生类中新增的数据成员。若有多个基类，则按照深度优先、从左到右的原则，根据继承列表中基类的定义顺序依次进行构造。在自定义构造函数时，可以通过成员初始化列表指定各基类的构造函数；对于编译器提供的派生类缺省构造函数，默认会使用基类的无参构造函数来构造基类子对象。派生类的析构过程与构造过程相反，先析构派生类中新增的数据成员，再析构基类子对象。例如：

```

1  #include <iostream>
2  using namespace std;
3  class Base {
4  public:
5      Base( int n ) : mNum( n ) {
6          ++mNum;
7          funcPriv( );
8      }
9      ~Base( ) {
10         --mNum;
11         funcPriv( );
12     }
13 protected:
14     void funcPriv( ) { cout << mNum << " "; }
15     int mNum;
16 };
17 class Derived : public Base {
18 public:
19     Derived( int n1, int n2 ) : Base( n1 ), mVal( n2 ) { }
20     ~Derived( ) { cout << mVal << " "; }
21 private:
22     int mVal;

```



```

23 };
24 int main( ) {
25     Derived d(1,99);
26     return 0;
27 }

```

运行结果:

```

2   99   1

```

这里，在实现派生类的构造函数时，通过 `Base(n1)`，指明使用 `Base` 类的单参构造函数构建基类子对象。对比下面的实现：

```

1   Derived( int n1, int n2 ) : mNum( n1 ), mVal( n2 ) { } //非法

```

这种写法非法且不合理。首先，基类子对象必须是完整的，只有通过基类构造函数才能保证子对象的完整性；其次，这种写法要求基类成员在派生类中必须是可访问的，此要求是难以满足的，例如基类中的私有成员在派生类中是不可访问的；最后，使用基类构造函数构建基类子对象有利于复用，例如修改了基类中的数据成员，派生类可不修改。

15.3.2 拷贝和赋值

由于派生类对象中包含基类子对象，在派生类的拷贝构造过程中，首先会先拷贝构造基类子对象，然后再拷贝派生类的新增数据成员。对于派生类的缺省拷贝构造函数，会自动使用基类的拷贝构造函数；对于自定义的拷贝构造函数，可以指定基类子对象的构造方法，即在拷贝构造函数的成员初始化列表中，显式地指明构造基类子对象所用的基类构造函数或基类拷贝构造函数，若成员初始化列表中没有显式指明，则使用基类的无参构造函数构造基类子对象。

对于赋值函数，过程也类似，应先进行基类子对象的赋值，然后再赋值派生的新增数据。在赋值基类子对象时，缺省赋值函数自动使用基类的赋值函数；自定义的赋值函数则需要显式地调用基类的赋值函数。例如：

```

1   #include <iostream>
2   class Base {
3   public:
4       Base( int num ) :mData( num ) { }
5       int getData( ) const { return mData; }
6       operator double( ) { return 0.6 * mData; }
7   private:
8       int mData;
9   };

```

```

10 class Derived :public Base {
11 public:
12     //using Base::Base;
13     Derived( int n1, int n2 ) : Base(n1),mNum( n2 ) { }
14     ~Derived( ) { }
15     Derived( const Derived & other ) :Base( other ) { mNum = other.mNum; }
16     Derived & operator=( const Derived & rhs ) {
17         if ( this != &rhs ) {
18             Base::operator=( rhs ); //显式调用基类的赋值函数
19             mNum = rhs.mNum;
20         }
21         return *this;
22     }
23     //或者
24     //Derived & operator=( Derived rhs ) {
25     //     Base::operator=( rhs ); //显式调用基类的赋值函数
26     //     std::swap( mNum, rhs.mNum );
27     //     return *this;
28     //}
29     void show( ) const {
30         std::cout << getData( ) << " " << mNum << std::endl;
31     }
32 private:
33     int mNum;
34 };
35 int main( )
36 {
37     Derived d1(1,2);
38     Derived d2(3,4);
39     Derived d3( d1 );
40     d3 = d2;
41     d1.show( ); //输出 1 2
42     d2.show( ); //输出 3 4
43     d3.show( ); //输出 3 4
44 }

```

15.4 覆盖和重写

这里说明几个概念：overload(重载)、newdefine(新定义)、redefine(重定义)、overwrite(覆盖)、override(重写或复写)。其中 override 详见虚函数。

15.4.1 重载(overload)

在同一个作用范围内，存在多个同名函数，这些函数名字相同，参数不同，virtual 关键字任意。

例如：

```
1  class MyClass {
2  public:
3      void f( );
4      void f( int )
5      void f( int ) const;
6      int  f( MyClass & other);
7      virtual A * f( int, int );//虚函数
8  };
```

15.4.2 新定义(newdefine)

在派生类中新增的函数，基类中无同名函数，virtual 关键字任意。例如：

```
1  class Base {
2  public:
3      void f( ) { }
4  };
5  class Derived: public Base {
6  public:
7      void g( ){ }          //newdefine
8      virtual int k( ) { } //newdefine
9  };
```

15.4.3 重定义(redefine)

派生类中的函数，与基类中函数同名，函数参数列表相同，virtual 关键字任意。例如：

```
1  class Base {
2  public:
3      void f( ) { }
4      virtual void f( int ) { }
5      void g( ) { }
6  };
7  class Derived: public Base {
8  public:
9      void f( ) { }          // 重定义了 Base::f( )
10     virtual void f( int ) { } // 重定义了 Base::f(int)
11     void g( ) { }          // 重定义了 Base::g( )
12 };
```

15.4.4 重写/复写(override)

重定义的特殊情况，即与基类中函数同名，函数参数列表相同，基类和派生类中都带 `virtual` 关键字。例如：

```

1  class Base {
2  public:
3      void f( ) { }
4      virtual void f( int ) { }
5      void g( ) { }
6  };
7  class Derived: public Base {
8  public:
9      void f( ) { }           // 重定义了 Base::f( )
10     virtual void f( int ) { } // 重定义了 Base::f(int)且重写了 Base::f(int)
11     void g( ) { }           // 重定义了 Base::g( )
12 };

```

15.4.5 覆盖(overwrite)

派生类中的一个或多个函数，与基类中函数同名，那么派生类中的函数将屏蔽(或称隐藏)基类中的所有同名函数，称为覆盖(overwrite)，也称隐藏(hide)。例如：

```

1  class Parent {
2  public:
3      void nvf( ) { /*略*/ }
4      void nvf( int ) { /*略*/ }
5      virtual Parent * vg( ) { /*略*/ }
6      virtual Parent * vg( int n ) { /*略*/ }
7  };
8  class Child :public Parent {
9  public:
10     void nvf(int n,int m) {
11         //Child::nvf(int,int)隐藏了基类中的 nvf( )和 nvf(int)
12         nvf( ); //非法
13         nvf( n ); //非法
14     }
15     virtual Child * vg( int n,int m ) {
16         //Child::vg(int,int)隐藏了基类中的 vg( )和 vg(int)
17         vg( ); //非法
18         vg(n); //非法
19     }
20 };

```

如果要使用被屏蔽(也称隐藏)的基类函数,可以显式指明或使用 `using` 语句。例如:

```

1  #include <iostream>
2  using namespace std;
3  class Parent {
4  public:
5      void nvf( ) { cout << "Parent::nvf( )\n"; }
6      void nvf( int ) { cout << "Parent::nvf(int)\n"; }
7      virtual Parent * vg( ) { cout << "Parent::vg( )\n"; return nullptr; }
8      virtual Parent * vg( int n ) { cout << "Parent::vg(int)\n"; return nullptr; }
9  };
10 class Child :public Parent {
11 public:
12     using Parent::nvf;    //使用 using 语句, 汇入了基类的 nvf( )和 nvf(int)
13     //using Parent::vg;    //可以试一试去掉注释
14
15     void nvf( int n, int m ) {
16         cout << "Child::nvf(int,int)\n";
17         nvf( ); //合法
18         nvf( n ); //合法
19     }
20     virtual Child * vg( int n, int m ) {
21         cout << "Child::vg(int,int)\n";
22         Parent::vg( ); //合法,显式指明调用 Parent::vg( ),但 vg( )不属于 Child 的接口函数
23         Parent::vg( n );//合法,显式指明调用 Parent::vg(int),但 vg(n)不属于 Child 的接口函数
24         return nullptr;
25     }
26 };
27 int main( ) {
28     Parent super;
29     Child sub;
30     cout << "1.-----\n";
31     super.nvf( );    //合法
32     super.nvf( 10 ); //合法
33     cout << "2.-----\n";
34     super.vg( );    //合法
35     super.vg( 2 );  //合法
36     cout << "3.-----\n";
37     sub.nvf( );      //合法
38     sub.nvf( 20 );   //合法
39     sub.nvf( 10, 20 );//合法
40     cout << "4.-----\n";
41     //sub.vg( );      //非法
42     //sub.vg( 3 );    //非法

```

```

43     sub.vg( 4, 5 );
44 }

```

15.4.6 最佳实践

由于继承的存在，在设计和定义类的成员函数时，应遵循以下条款：

1. 不要重定义(redefine)基类的非虚函数。一般认为，如果重定义了基类的非虚函数，那么本质上破坏了基类的不变性，违背抽象数据类型的不变性要求。重定义中的基类函数只有两种，一种是非虚函数，另一种是虚函数。基于此条款，只应重定义基类的虚函数，也就是重写(override)。这也是很少说重定义，而经常说重写的原因。
2. 派生类中重写(override)基类的一个成员函数时，应确保参数列表一致，返回值的类型相同或相容。详见虚函数。
3. 慎用函数重载。如果基类中有多个重载函数，并且在派生类中定义了同名函数，那么，不论函数是否为虚函数，派生类中的同名函数都将隐藏掉基类的多个同名函数。这样，即使是公有继承，派生类也将不再具有基类的接口，不再满足子类"是一个"或"是一种"父类的含义；另外，定义一个类时，不知道未来是否会被继承，若类中有重载函数，而且派生的子类中定义了一个同名函数，则隐含要求子类应该显式地汇入基类同名函数，或者重定义或重写基类的所有重载的同名函数，这很难保证。

15.5 继承和组合

复用一类可以使用黑盒复用，也可以使用白盒复用。黑盒复用的过程中可以通过定义类(对象)间的水平关系，如依赖、关联、组合、聚合等实现既有类的功能复用；白盒复用则是使用类的继承机制实现既有类的代码复用。在复用类时，如果没有既有类的源码，那么只能采用黑盒复用的方式；如果有源码，选择黑盒复用还是白盒复用呢？回答此问题，需要先理解继承的逻辑含义。继承中有三种继承方式：public、protected 和 private，每种继承方式表示的逻辑含义是不同的。

15.5.1 public 继承

public 继承方式下，也称基类为父类，派生类为子类。子类和父类之间具有是"is a"或"like a"或"is a kind of"的关系，也就是公有继承表明的是"子类是一种父类"或"子类对象是一个父类对象"或"子类象一种父类"的含义。例如：水果作为父类，苹果和橘子可作为子类，因为说"苹果是一种水果，橘子是一种水果"是合理的，符合人的正常认知；相应地，如果两个类之间不具有类似的含义，就不应该使用 public 继承。例：

```

1  class Car {
2  public:
3      void run( );
4      void brake( );
5  protected:
6      int addWeight( int w );
7      int reduceWeight( int w );
8  private:
9      int mWeight;
10     int mWheels;
11 };
12 class MTCar :public Car {
13 public:
14     void funcMT( );
15 protected:
16     /* 略 */
17 private:
18     /* 略 */
19 };
20 class ATCar :public Car {
21 public:
22     void funcAT( );
23 protected:
24     /* 略 */
25 private:
26     /* 略 */
27 };

```

从逻辑上手动挡轿车(MTCar)是一种轿车(Car),自动挡轿车(ATCar)也是一种轿车(Car),符合公有继承的含义;从实现上,MTCar 类的公有行为能力相比 Car 类只多不少,ATCar 类也一样,这正是逻辑学中分类的概念。因此,在公有继承时,从父类派生子类,实际是对父类类型进行子类型化的过程,而且这种类型和子类型的关系是无法用依赖、关联、组合或聚合关系表示的。基于类型及子类型的逻辑含义,在程序中所有用到父类对象的地方,应该都可以用子类对象代替,这正是使用公有继承的价值所在。例如:

```

1  class Car { /* 略 */ };
2  class MTCar : public Car { /* 略 */ };
3  class ATCar : public Car { /* 略 */ };
4  class Client {
5  public:
6      Client( Car & c) { mpCar = &c; } //实参 c 可以是 Car 的任意子类对象
7      void func( Car & c) { c.run( ); } //实参 c 可以是 Car 的任意子类对象

```

```

8  private:
9      Car * mpCar; //类型为 Car *, 实际对象可以是 Car 的任意子类对象
10 };
11 int main( ) {
12     Car car;
13     MTCar mtc;
14     ATCar atcar;
15
16     Client mycar(atcar);    //形参类型为 Car&, 实参对象的类型为 ATCar
17     mycar.func(mtc);       //形参类型为 Car&, 实参对象的类型为 MTCar
18 }

```

15.5.2 private 继承

私有继承表示派生类与基类之间存在“has-a”、“contain-a”或“implement-in-terms-of”的关系。例如，在 `class Derived : private Base { }` 中，意味着 `Derived` 类对象内部包含一个 `Base` 类对象，或者 `Derived` 对象的实现依赖于 `Base` 类对象。这种方式使得 `Derived` 类可以使用 `Base` 类的功能，但不会公开 `Base` 类的接口。例如：

```

1  class Bike {
2  public:
3      void move( );
4      void stop( );
5      void repair( );
6  private:
7      int changeColor(int );
8  private:
9      int mColor;
10 };
11 class Player : private Bike {
12 public:
13     void startRace( ) { move( ); }
14     void endRace( ) { stop( ); }
15 protected:
16     int curStrength( ) const;
17 private:
18     int mStrength;
19     int mAge;
20 };

```

在私有继承下，`Player` 类的公有成员与 `Bike` 类的公有成员没有交集，这意味着 `Player` 类和 `Bike` 类在行为能力上完全不同。`Player` 类私有继承 `Bike` 类的目的仅仅是为了在实现 `Player` 类的成员函

数时能够利用 Bike 类的功能。

15.5.3 protected 继承

保护继承和私有继承都表示派生类与基类之间存在 "has-a"、"contain-a" 或 "implement-of" 的关系。两者的区别在于这种关系的传递性：在私有继承中，这种关系仅限于直接的派生类与基类之间，不会传递给更远的派生类。而在保护继承中，这种关系可以沿着继承树传递给所有间接派生类，而不仅仅局限于直接派生类。例如：

```

1  class A {
2  public:
3      void funcPubA( );
4  protected:
5      void funcProtA( );
6  private:
7      void funcPrivA( );
8  };
9  class B : protected A {
10 public:
11     void funcPubB( );
12 protected:
13     void funcProtB( );
14 private:
15     void funcPrivB( );
16 };
17 class C : protected B {
18 public:
19     void funcPubC( );
20 protected:
21     void funcProtC( );
22 private:
23     void funcPrivC( );
24 };

```

A、B、C 三个类的公有接口完全不同，类 B 有保护成员 A::funcPubA、A::funcProtA、B::funcProtB，类 C 有保护成员 A::funcPubA、A::funcProtA、B::funcPubB、B::funcProtB、C::funcProtC，可见，在保护继承下，类 A 中的公有和保护成员可以沿继承链传递给所有派生类，因此间接派生的 C 类与顶层基类 A 具有类似私有继承的逻辑关系。

15.5.4 组合优先

在复用类时，可以选择黑盒复用或白盒复用。黑盒复用是指通过依赖、关联、组合和聚合关系，在新定义的类中复用已有类的功能，以实现类的扩展。而白盒复用则是通过继承关系，从已有类派生出新类，直接复用其代码来实现扩展。

选择复用方式时应遵循以下原则：

- (1) 若无法获取类的源代码，则只能采用黑盒复用。
- (2) 优先考虑组合而非继承。应避免使用保护继承和私有继承，当遇到这两种继承情况时，建议用组合关系替代。这里需要明确使用组合关系，是因为组合能更好地体现派生类包含基类子对象的关系，并且二者生命周期一致，只有组合关系能够准确表达这种包含性。因此，现代面向对象程序设计语言通常仅支持公有继承，不再提供保护和私有继承。
- (3) 在需要子类型化时使用公有继承。当要将类型细分为更多子类型时，才应使用公有继承。因为只有公有继承才能明确表达“子类是一种父类”的逻辑关系，而依赖、关联、组合和聚合等水平关系则不具备这种语义。

例如，之前提到的使用私有继承的 `Player` 类示例，可以改用组合关系来实现。

```

1  class Bike {
2  public:
3      void move( );
4      void stop( );
5      void repair( );
6  private:
7      int changeColor(int n );
8  private:
9      int mColor;
10 };
11 class Player {
12 public:
13     void startRace( )    { mBike.move( ); }
14     void endRace( )      { mBike.stop( ); }
15 protected:
16     int curStrength( ) const;
17 private:
18     void move( )         { mBike.move( ); }
19     void stop( )         { mBike.stop( ); }
20     void repair( )       { mBike.repair( ); }
21 private:
22     Bike    mBike;
23     int     mStrength;
```

```

24     int    mAge;
25 };

```

在使用组合替代继承时，必须确保 `Player` 类的可访问成员仍然保持其可访问性。例如，在继承的情况下，派生类 `Player` 拥有公有接口 `Player::startRace` 和 `Player::endRace`，以及私有方法 `move`、`stop` 和 `repair`。当改为使用组合时，需保持公有接口的可访问性。

这里，`Bike` 类作为 `Player` 类的一个子对象存在，这不仅体现了组合关系，还使得 `Player` 可以使用缺省的析构、拷贝和赋值函数。当然，你也可以选择其他方式将 `Bike` 类作为数据成员嵌入到 `Player` 中，但此时需要注意，可能需要自定义实现构造、析构、拷贝和赋值函数。例如：

```

1  class Player {
2  public:
3      Player( ) : mpBike( new Bike ) { }
4      ~Player( ) { delete mpBike; }
5      Player( const Player & rhs ) : mpBike( new Bike( *rhs.mpBike ) )
6                                     , mStrength( rhs.mStrength ), mAge( rhs.mAge ){ }
7      Player & operator=( Player rhs ) {
8          swap( mpBike, rhs.mpBike );
9          swap( mStrength, rhs.mStrength );
10         swap( mAge, rhs.mAge );
11         return *this;
12     }
13     void startRace( ) { mpBike->move( ); }
14     void endRace( ) { mpBike->stop( ); }
15 protected:
16     int curStrength( ) const { return mStrength; }
17 private:
18     void move( )      { mpBike->move( ); }
19     void stop( )      { mpBike->stop( ); }
20     void repair( )    { mpBike->repair( ); }
21 private:
22     Bike *   mpBike;
23     int      mStrength = 0;
24     int      mAge = 0;
25 };

```

进一步地，如果 `Bike` 类中的 `int changeColor(int n);` 的访问控制符是 `protected`，那么我们如何通过组合的方式改写 `Player` 类呢？请自行实现。